

NOM et Prénom: _____

Consignes

- **Durée** : 1h30;
- 1 feuille A4 R/V autorisée et calculatrice interdite;
- Vous pouvez répondre aux problèmes dans l'ordre que vous souhaitez;
- Rendre le sujet à la fin avec la copie;

1 Question aux Choix Multiples

Répondez en cochant **le ou les choix correct(s)**. Notation avec points négatifs :

1 points correct, 0 point si pas de réponse, -0.5 pt en cas d'erreur

1. Supposons que j'ai deux variables `mot1` et `mot2` de type `char*`, quel pourrait être le prototype de la fonction `concat` qui prends en entrée ces variables et les concatène à l'adresse de la variable `mot1` ? (1)

- ☐ `void concat(char* mot1, char* mot2)`
- ☐ `char* concat(char* mot1, char* mot2)`
- ☐ `char* concat(char* x, char* y)`
- ☐ Aucune de ces réponses

2. Soit le code suivant: (1)

```
1 #include <stdio.h>
2
3 int main()
4 {
5     float c = 5.0;
6     printf ("Temperature in Fahrenheit is %.2f", (9/5)*c + 32);
7     return 0;
8 }
```

Quelle est la sortie affichée sur le terminal ?

- ☐ Temperature in Fahrenheit is 41.00
- ☐ Temperature in Fahrenheit is 37.00
- ☐ Temperature in Fahrenheit is 0.00
- ☐ Il y a une erreur de compilation

3. Pour allouer dynamiquement un espace en mémoire, quelles(s) fonctions peut-on utiliser ? (1)

☐ `allocation` ☐ `malloc` ☐ `calloc` ☐ `realloc` ☐ Aucune de ces fonctions

4. Soit le programme: (1)

```
1 #include <stdio.h>
2 int main()
3 {
4     int n;
5     for (n = 9; n!=20; n+2)
6         printf("%d ", n);
7     return 0;
8 }
```

Quelle est la sortie du programme ?

- ☐ 9 11 13 15 17 19
- ☐ 9 11 13 15 17 19 21
- ☐ Aucune de ces réponses là
- ☐ Il y a une erreur de compilation

5. Soit le programme:

(1)

```
1 int main()  
2 {  
3     int x = 5;  
4     int *p = &x;  
5     p++;  
6 }
```

En supposant une architecture 32 bits où l'adresse des variables commence à 0x01AF en hexadécimal, qu'elle est la valeur de p à la fin ?

- ☐ On ne sait pas
- ☐ 0x01B0
- ☐ 0x01B3
- ☐ 6

2 Numération

On s'intéressera dans cet exercice à la réalisation d'une base de donnée pour accueillir des informations issues de capteurs atmosphériques placé sur des station en altitude. Les informations recueillies par le capteur sont les suivantes:

- le numéro du capteur compris entre 0 et 170.
- la date de la mesure dans une chaîne de caractère au format "JJ-MM-AA_hh-mm-ss" (jour, mois, année, heure, minutes, secondes).
- la pression atmosphérique qui est un nombre réel codé sur 32 bits.
- la température qui est un nombre entier compris entre -60 et +60.

1. On suppose pour ne pas alourdir les types, que la chaîne de caractère de la date est codée d'une façon où chaque lettre est codée comme une lettre ASCII. Combien de bits sont alors nécessaires pour coder la chaîne de caractère de la date ? (2)

2. Combien de bits sont alors nécessaires pour coder toutes les informations issues du capteur de manière simple sans codage particulier ? (1)

3. En supposant que l'on code la température sur 8 bits avec une représentation des nombres signés en complément à 2. À quelle température correspond le code 1101 0001 ? (1)

4. En supposant que l'on trouve un code binaire qui représente une entrée de la base de donnée sur 256 bits. Sachant qu'il y a une mesure faite toutes les secondes et que le disque dur interne au capteur possède une capacité de 1024 Go ($1024 \text{ Go} = 1024 \times 10^9 \text{ octets}$), combien de jours faudrait-il au capteur pour saturer toute la mémoire disponible? (1)

3 Où l'on parle de perceptron

Le perceptron est l'un des premiers algorithmes d'apprentissage automatique développé en 1958 par Frank Rosenblatt. Inspiré des neurones biologiques, il permet de résoudre des problèmes de classification linéaire en séparant les données en deux classes. Le perceptron repose sur une règle de décision basée sur une combinaison linéaire pondérée des entrées et un biais. Cet algorithme, bien qu'initialement simple, a marqué le début de la recherche sur les réseaux neuronaux et a jeté les bases des approches modernes en apprentissage automatique.

Supposons un problème où l'on veut savoir étant donné la pression et température d'une cocotte minute, si celle-ci est dangereuse. On décide à partir de l'équation de décision du perceptron qui est donnée par :

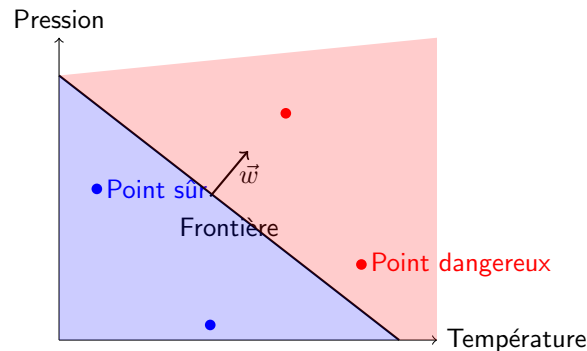
$$y = \text{sign}(\vec{w} \cdot \vec{x} + b)$$

où :

- $\vec{w}$ est le vecteur des poids,
- $\vec{x}$ est le vecteur des entrées (pression, température)
- b est le biais,
- sign est une fonction qui retourne $+1$ si l'entrée est positive et -1 sinon.
- $\cdot$ désigne le produit scalaire

Dans ce cadre, la valeur de sortie $y = +1$ signifie que la cocotte est dangereuse, et $y = -1$ non.

Illustration graphique



Dans ce graphique, la zone en rouge hachurée correspond à un état dangereux, et la zone en bleu rayée correspond à un état sûr.

3.1 Partie 1 : Implémentation de la règle de décision

Dans un premier temps, on suppose $\vec{w}$ et b connus et on veut juste savoir si la mesure que l'on fait est dans une zone sûre.

1. Définir deux structures `Point_st` et `Vec2_st`, permettant de représenter un point (coordonnées 2D) et un vecteur 2D. Donner un exemple de programme `main` qui initialise et utilise ces structures. (2)
2. Implémenter une fonction `sign` qui prend en paramètre un nombre réel et retourne son signe. (1)
3. Implémenter une fonction `perceptron_rule` qui applique la règle de décision pour un point donné en utilisant un vecteur de poids w , un biais b , et retourne $+1$ ou -1 . (1)
4. En supposant que `Point_st` et `Vec2_st` sont définis dans un fichier `data.h` et que les fonctions précédentes sont implémentées dans `data.c`, écrire un fichier `main.c` qui utilise ces éléments. Le programme doit lire à partir de l'entrée utilisateur un vecteur de poids $\vec{w}$, un biais b , et les coordonnées d'un point, puis afficher si la cocotte est sûre ou dangereuse. (2)
5. Donner la commande de compilation pour produire un exécutable nommé `cocotte_tobeornottobe`. (1)

3.2 Partie 2 : Apprentissage des poids et du biais

On suppose maintenant que pour notre cocotte, les paramètres $\vec{w}$ et b sont inconnus et l'on fait des essais (en environnement contrôlé) pour les estimer. On a alors un ensemble de données $\{\vec{x}_1, \dots, \vec{x}_n\}$ et des labels associés $\{y_1, \dots, y_n\}$ (qui sont des valeurs de +1 ou bien -1).

On peut alors apprendre les poids à l'aide d'un algorithme itératif à partir d'une supposition initiale $\vec{w}_{(0)}$, $b_{(0)}$ et en itérant sur toutes les données avec la formule, pour une donnée $\vec{x}_i, y_i$:

$$\vec{w}^{(i+1)} = \vec{w}^{(i)} + \eta y_i \vec{x}_i,$$

$$b^{(i+1)} = b^{(i)} + \eta y_i,$$

où η est un paramètre qui indique à quel point on change les valeurs apprises appelé, le *learning rate*.

On suppose que l'on a un fichier `donnees.txt` où l'on stocke les valeurs mesurées et si la cocotte a explosé ou pas.

1. En supposant qu'une fonction `int compter_points(char *filename)` retourne le nombre de points dans le fichier de données, écrire une fonction `allouer_train_points` qui alloue dynamiquement un tableau de points pour stocker ces données et retourne son adresse. Faire aussi une fonction `allouer_labels` qui alloue un tableau d'entiers pour les labels. (2)
2. Écrire une fonction `update_w` qui met à jour le vecteur de poids `w` en utilisant un point donné selon la règle de mise à jour du perceptron en dépendant du paramètre η . (2)
3. Écrire une fonction `update_b` qui met à jour le biais `b` en utilisant un point donné selon la règle de mise à jour du perceptron en dépendant du paramètre η . (1)
4. En supposant qu'une fonction `void read_data(char *filename, Point_st *tab_points, int* tab_labels)` lit les données d'un fichier et les stocke dans les tableau alloués en dehors de la fonction, écrire une fonction `main` qui effectue l'entraînement du perceptron. Le programme doit afficher les nouvelles valeurs de `w` et `b` après chaque point d'entraînement. Pour les valeurs initiales, on pourra prendre le vecteur nul $\vec{w}^{(0)} = (0, 0)$ et $b = 0$. (3)